

Министерство науки и высшего образования РФ
Пензенский государственный университет
Кафедра «Вычислительная техника»

Отчет
по лабораторной работе № 5
по курсу «Логика и основы алгоритмизации в инженерных задачах»
на тему «Определение характеристик графов»

Выполнил
студент группы 24ВВВ2:

Воеводин И.А.
Гуляевский Д.С.
Каташов К.А.

Приняли
Юрова О.В.
Деев М.В.

Пенза, 2025

Цель работы

Определение характеристик графов.

Лабораторное задание

Задание 1

1. Сгенерируйте (используя генератор случайных чисел) матрицу смежности для неориентированного графа G . Выведите матрицу на экран.
2. Определите размер графа G , используя матрицу смежности графа.
3. Найдите изолированные, концевые и доминирующие вершины.

Задание 2

1. Постройте для графа G матрицу инцидентности.
2. Определите размер графа G , используя матрицу инцидентности графа.
3. Найдите изолированные, концевые и доминирующие вершины.

Описание метода решения

В ходе выполнения работы для анализа неориентированного графа были использованы два альтернативных представления: матрица смежности и матрица инцидентности. Оба подхода начинаются с генерации случайного графа путём создания симметричной матрицы смежности, где наличие ребра между вершинами определяется случайным образом. На основе этой матрицы либо непосредственно выполняется анализ (в первом задании), либо строится матрица инцидентности (во втором задании).

Ключевой этап анализа — вычисление степеней вершин. Для матрицы смежности степень вершины определяется как сумма элементов в соответствующей строке, а для матрицы инцидентности — как сумма элементов строки, соответствующей вершине. Размер графа (количество рёбер) в случае матрицы смежности вычисляется подсчётом единиц выше главной диагонали, а для матрицы инцидентности равен количеству столбцов.

На основе полученных степеней вершин производится классификация: изолированные вершины имеют степень 0, концевые — степень 1, доминирующие — степень $n-1$, где n — количество вершин в графе. Все

результаты, включая визуальное представление матриц, размер графа и классификацию вершин, выводятся на экран. Оба метода подтверждают эквивалентность различных представлений графа для решения задач структурного анализа.

Листинг

Задание 1

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <locale.h>

//функция для генерации матрицы
void generateMatrix(int n, int** matrix) {
    for (int i = 0; i < n; i++) {
        for (int j = i; j < n; j++) {

            if (rand() % 2 == 1) {
                matrix[i][j] = 1;
                matrix[j][i] = 1;
            }
        }
    }
}

//функция для вывода
void printMatrix(int n, int** matrix) {
    printf("Матрица смежности:\n ");
    for (int i = 0; i < n; i++) {
        printf("%2d ", i + 1);
    }
    printf("\n");

    for (int i = 0; i < n; i++) {
        printf("%2d ", i + 1);
        for (int j = 0; j < n; j++) {
            printf("%2d ", matrix[i][j]);
        }
        printf("\n");
    }
}

//функция для степени
int calculateDegree(int n, int** matrix, int vertex) {
    int degree = 0;
    for (int i = 0; i < n; i++) {
        degree += matrix[vertex][i];
    }
    return degree;
}

//определение размера графа
int graphSize(int n, int** matrix) {
    int edges = 0;
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            edges += matrix[i][j];
        }
    }
    return edges;
}
```

```
}
```

```
//изолированные вершины
```

```
void findIsolated(int n, int** matrix) {  
    printf("Изолированные вершины (степень 0): ");  
    int found = 0;  
    for (int i = 0; i < n; i++) {  
        if (calculateDegree(n, matrix, i) == 0) {  
            printf("%d ", i + 1);  
            found = 1;  
        }  
    }  
    if (!found) printf("нет");  
    printf("\n");  
}
```

```
//концевые вершин
```

```
void findEnd(int n, int** matrix) {  
    printf("Концевые вершины (степень 1): ");  
    int found = 0;  
    for (int i = 0; i < n; i++) {  
        if (calculateDegree(n, matrix, i) == 1) {  
            printf("%d ", i + 1);  
            found = 1;  
        }  
    }  
    if (!found) printf("нет");  
    printf("\n");  
}
```

```
//доминирующие вершины
```

```
void findDominating(int n, int** matrix) {  
    printf("Доминирующие вершины (степень %d): ", n - 1);  
    int found = 0;  
    for (int i = 0; i < n; i++) {  
        if (calculateDegree(n, matrix, i) == n - 1) {  
            printf("%d ", i + 1);  
            found = 1;  
        }  
    }  
    if (!found) printf("нет");  
    printf("\n");  
}
```

```
//функция для вывода степеней вершин
```

```
void printDegrees(int n, int** matrix) {  
    printf("Степени вершин:\n");  
    for (int i = 0; i < n; i++) {  
        printf("Вершина %d: степень %d\n", i + 1, calculateDegree(n, matrix, i));  
    }  
}
```

```
int main() {  
    srand(time(NULL));  
    setlocale(LC_ALL, "rus");
```

```

int n;

printf("Введите количество вершин графа: ");
scanf_s("%d", &n);

// выделение памяти
int** matrix = (int**)malloc(n * sizeof(int*));
for (int i = 0; i < n; i++) {
    matrix[i] = (int*)calloc(n, sizeof(int));
}

generateMatrix(n, matrix);

printf("\n");
printMatrix(n, matrix);

int size = graphSize(n, matrix);
printf("\nРазмер графа (количество ребер): %d\n", size);

printf("\n");
printDegrees(n, matrix);

printf("\n");
findIsolated(n, matrix);
findEnd(n, matrix);
findDominating(n, matrix);

//освобождение памяти
for (int i = 0; i < n; i++) {
    free(matrix[i]);
}
free(matrix);

return 0;
}

```

Задание 2

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <locale.h>

// Функция для генерации матрицы
int** generateIncidenceMatrix(int n, int* edge_count) {
    // Создаем временную матрицу смежности для генерации графа
    int** adj_matrix = (int**)malloc(n * sizeof(int*));
    for (int i = 0; i < n; i++) {
        adj_matrix[i] = (int*)calloc(n, sizeof(int));
    }

    // Генерируем ребра
    *edge_count = 0;
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {

```

```

        if (rand() % 2 == 1) {
            adj_matrix[i][j] = 1;
            adj_matrix[j][i] = 1;
            (*edge_count)++;
        }
    }
}

// Создаем матрицу инцидентности
int** inc_matrix = (int**)malloc(n * sizeof(int*));
for (int i = 0; i < n; i++) {
    inc_matrix[i] = (int*)calloc(*edge_count, sizeof(int));
}

// Заполняем
int edge_index = 0;
for (int i = 0; i < n; i++) {
    for (int j = i + 1; j < n; j++) {
        if (adj_matrix[i][j] == 1) {
            inc_matrix[i][edge_index] = 1;
            inc_matrix[j][edge_index] = 1;
            edge_index++;
        }
    }
}

// Освобождаем временную матрицу смежности
for (int i = 0; i < n; i++) {
    free(adj_matrix[i]);
}
free(adj_matrix);

return inc_matrix;
}

// Функция для вывода матрицы
void printIncidenceMatrix(int n, int edge_count, int** matrix) {
    printf("Матрица инцидентности:\n ");
    for (int j = 0; j < edge_count; j++) {
        printf("e%-2d", j + 1);
    }
    printf("\n");

    for (int i = 0; i < n; i++) {
        printf("v%-2d", i + 1);
        for (int j = 0; j < edge_count; j++) {
            printf("%2d ", matrix[i][j]);
        }
        printf("\n");
    }
}

// Функция для вычисления степени вершины через матрицу инцидентности
int calculateDegree(int n, int edge_count, int** matrix, int vertex) {
    int degree = 0;
    for (int j = 0; j < edge_count; j++) {

```

```

        degree += matrix[vertex][j];
    }
    return degree;
}

```

```

int graphSize(int edge_count) {
    return edge_count;
}

```

// Функция для поиска изолированных вершин

```

void findIsolatedVertices(int n, int edge_count, int** matrix) {
    printf("Изолированные вершины (степень 0): ");
    int found = 0;
    for (int i = 0; i < n; i++) {
        if (calculateDegree(n, edge_count, matrix, i) == 0) {
            printf("%d ", i + 1);
            found = 1;
        }
    }
    if (!found) printf("нет");
    printf("\n");
}

```

// Функция для поиска концевых вершин

```

void findEndVertices(int n, int edge_count, int** matrix) {
    printf("Концевые вершины (степень 1): ");
    int found = 0;
    for (int i = 0; i < n; i++) {
        if (calculateDegree(n, edge_count, matrix, i) == 1) {
            printf("%d ", i + 1);
            found = 1;
        }
    }
    if (!found) printf("нет");
    printf("\n");
}

```

// Функция для поиска доминирующих вершин

```

void findDominatingVertices(int n, int edge_count, int** matrix) {
    printf("Доминирующие вершины (степень %d): ", n - 1);
    int found = 0;
    for (int i = 0; i < n; i++) {
        if (calculateDegree(n, edge_count, matrix, i) == n - 1) {
            printf("%d ", i + 1);
            found = 1;
        }
    }
    if (!found) printf("нет");
    printf("\n");
}

```

// Функция для вывода степеней всех вершин

```

void printVertexDegrees(int n, int edge_count, int** matrix) {
    printf("Степени вершин:\n");
    for (int i = 0; i < n; i++) {

```



```

        printf("Вершина %d: степень %d\n", i + 1, calculateDegree(n, edge_count, matrix, i));
    }
}

int main() {
    srand(time(NULL));
    setlocale(LC_ALL, "rus");

    int n;
    int edge_count;

    printf("Введите количество вершин графа: ");
    scanf_s("%d", &n);

    int** inc_matrix = generateIncidenceMatrix(n, &edge_count);

    printf("\n");
    printIncidenceMatrix(n, edge_count, inc_matrix);

    int size = graphSize(edge_count);
    printf("\nРазмер графа (количество ребер): %d\n", size);

    printf("\n");
    printVertexDegrees(n, edge_count, inc_matrix);

    printf("\n");
    findIsolatedVertices(n, edge_count, inc_matrix);
    findEndVertices(n, edge_count, inc_matrix);
    findDominatingVertices(n, edge_count, inc_matrix);

    // Освобождение памяти
    for (int i = 0; i < n; i++) {
        free(inc_matrix[i]);
    }
    free(inc_matrix);

    return 0;
}

```

Псевдокод

1-е задание

ПРОГРАММА АнализГрафаМатрицаСмежности

ПЕРЕМЕННЫЕ

n: ЦЕЛОЕ // Количество вершин
matrix: ЦЕЛОЙ[][] // Матрица смежности

ФУНКЦИЯ generateMatrix(n: ЦЕЛОЕ, matrix: ЦЕЛОЙ[][])

ДЛЯ i ОТ 0 ДО n-1

ДЛЯ j ОТ i ДО n-1

ЕСЛИ случайное_число() % 2 == 1 ТО

matrix[i][j] = 1

matrix[j][i] = 1

КОНЕЦ ЕСЛИ
КОНЕЦ ЦИКЛА
КОНЕЦ ЦИКЛА

ФУНКЦИЯ printMatrix(n: ЦЕЛОЕ, matrix: ЦЕЛОЙ[][])

ВЫВЕСТИ "Матрица смежности:"

ВЫВЕСТИ " "

ДЛЯ i ОТ 0 ДО n-1

ВЫВЕСТИ форматированный вывод (i+1)

ВЫВЕСТИ перевод строки

ДЛЯ i ОТ 0 ДО n-1

ВЫВЕСТИ форматированный вывод (i+1)

ДЛЯ j ОТ 0 ДО n-1

ВЫВЕСТИ форматированный вывод matrix[i][j]

ВЫВЕСТИ перевод строки

ФУНКЦИЯ calculateDegree(n: ЦЕЛОЕ, matrix: ЦЕЛОЙ[][], vertex: ЦЕЛОЕ): ЦЕЛОЕ

degree = 0

ДЛЯ i ОТ 0 ДО n-1

degree = degree + matrix[vertex][i]

ВЕРНУТЬ degree

ФУНКЦИЯ graphSize(n: ЦЕЛОЕ, matrix: ЦЕЛОЙ[][]): ЦЕЛОЕ

edges = 0

ДЛЯ i ОТ 0 ДО n-1

ДЛЯ j ОТ i+1 ДО n-1

edges = edges + matrix[i][j]

ВЕРНУТЬ edges

ФУНКЦИЯ findIsolated(n: ЦЕЛОЕ, matrix: ЦЕЛОЙ[][])

ВЫВЕСТИ "Изолированные вершины (степень 0): "

found = ЛОЖЬ

ДЛЯ i ОТ 0 ДО n-1

ЕСЛИ calculateDegree(n, matrix, i) == 0 ТО

ВЫВЕСТИ (i+1)

found = ИСТИНА

КОНЕЦ ЕСЛИ

КОНЕЦ ЦИКЛА

ЕСЛИ НЕ found ТО ВЫВЕСТИ "нет"

ВЫВЕСТИ перевод строки

ФУНКЦИЯ findEnd(n: ЦЕЛОЕ, matrix: ЦЕЛОЙ[][])

ВЫВЕСТИ "Концевые вершины (степень 1): "

found = ЛОЖЬ

ДЛЯ i ОТ 0 ДО n-1

ЕСЛИ calculateDegree(n, matrix, i) == 1 ТО

ВЫВЕСТИ (i+1)

found = ИСТИНА

КОНЕЦ ЕСЛИ

КОНЕЦ ЦИКЛА

ЕСЛИ НЕ found ТО ВЫВЕСТИ "нет"

ВЫВЕСТИ перевод строки

ФУНКЦИЯ findDominating(n: ЦЕЛОЕ, matrix: ЦЕЛОЙ[][])

ВЫВЕСТИ "Доминирующие вершины (степень " + (n-1) + "): "

```

found = ЛОЖЬ
ДЛЯ i ОТ 0 ДО n-1
    ЕСЛИ calculateDegree(n, matrix, i) == n-1 ТО
        ВЫВЕСТИ (i+1)
        found = ИСТИНА
    КОНЕЦ ЕСЛИ
КОНЕЦ ЦИКЛА
ЕСЛИ НЕ found ТО ВЫВЕСТИ "нет"
ВЫВЕСТИ перевод строки

```

```

ФУНКЦИЯ printDegrees(n: ЦЕЛОЕ, matrix: ЦЕЛОЙ[[]])
    ВЫВЕСТИ "Степени вершин:"
    ДЛЯ i ОТ 0 ДО n-1
        ВЫВЕСТИ "Вершина " + (i+1) + ": степень " + calculateDegree(n, matrix, i)

```

ОСНОВНАЯ ПРОГРАММА

```

УСТАНОВИТЬ случайный генератор
УСТАНОВИТЬ локаль

```

```

ВЫВЕСТИ "Введите количество вершин графа: "
ВВЕСТИ n С КЛАВИАТУРЫ

```

```

// Выделение памяти для матрицы
matrix = ВЫДЕЛИТЬ ПАМЯТЬ(ЦЕЛОЙ[n][n])
ДЛЯ i ОТ 0 ДО n-1
    matrix[i] = ВЫДЕЛИТЬ ПАМЯТЬ(ЦЕЛОЙ[n])
// Инициализация нулями
ДЛЯ j ОТ 0 ДО n-1
    matrix[i][j] = 0

```

```

generateMatrix(n, matrix)

```

```

ВЫВЕСТИ пустую строку
printMatrix(n, matrix)

```

```

size = graphSize(n, matrix)
ВЫВЕСТИ "Размер графа (количество ребер): " + size

```

```

ВЫВЕСТИ пустую строку
printDegrees(n, matrix)

```

```

ВЫВЕСТИ пустую строку
findIsolated(n, matrix)
findEnd(n, matrix)
findDominating(n, matrix)

```

```

// Освобождение памяти
ДЛЯ i ОТ 0 ДО n-1
    ОСВОБОДИТЬ ПАМЯТЬ(matrix[i])
ОСВОБОДИТЬ ПАМЯТЬ(matrix)

```

```

ВЕРНУТЬ 0

```

КОНЕЦ ПРОГРАММЫ

2-е задание

ПРОГРАММА АнализГрафа

ПЕРЕМЕННЫЕ

```
n: ЦЕЛОЕ           // Количество вершин
edge_count: ЦЕЛОЕ   // Количество ребер
inc_matrix: ЦЕЛОЙ[][] // Матрица инцидентности
```

ФУНКЦИЯ generateIncidenceMatrix(n: ЦЕЛОЕ, edge_count: ССЫЛКА НА ЦЕЛОЕ): ЦЕЛОЙ[][]

```
// Создаем временную матрицу смежности
adj_matrix = ВЫДЕЛИТЬ ПАМЯТЬ(ЦЕЛОЙ[n][n])
```

```
// Инициализируем матрицу смежности нулями
```

```
ДЛЯ i ОТ 0 ДО n-1
```

```
    ДЛЯ j ОТ 0 ДО n-1
```

```
        adj_matrix[i][j] = 0
```

```
// Генерируем случайные ребра
```

```
edge_count = 0
```

```
ДЛЯ i ОТ 0 ДО n-1
```

```
    ДЛЯ j ОТ i+1 ДО n-1
```

```
        ЕСЛИ случайное_число() % 2 == 1 ТО
```

```
            adj_matrix[i][j] = 1
```

```
            adj_matrix[j][i] = 1
```

```
            edge_count = edge_count + 1
```

```
// Создаем матрицу инцидентности
```

```
inc_matrix = ВЫДЕЛИТЬ ПАМЯТЬ(ЦЕЛОЙ[n][edge_count])
```

```
// Инициализируем матрицу инцидентности нулями
```

```
ДЛЯ i ОТ 0 ДО n-1
```

```
    ДЛЯ j ОТ 0 ДО edge_count-1
```

```
        inc_matrix[i][j] = 0
```

```
// Заполняем матрицу инцидентности
```

```
edge_index = 0
```

```
ДЛЯ i ОТ 0 ДО n-1
```

```
    ДЛЯ j ОТ i+1 ДО n-1
```

```
        ЕСЛИ adj_matrix[i][j] == 1 ТО
```

```
            inc_matrix[i][edge_index] = 1
```

```
            inc_matrix[j][edge_index] = 1
```

```
            edge_index = edge_index + 1
```

```
// Освобождаем временную матрицу
```

```
ОСВОБОДИТЬ ПАМЯТЬ(adj_matrix)
```

```
ВЕРНУТЬ inc_matrix
```

ФУНКЦИЯ printIncidenceMatrix(n: ЦЕЛОЕ, edge_count: ЦЕЛОЕ, matrix: ЦЕЛОЙ[][])

```
ВЫВЕСТИ "Матрица инцидентности:"
```

```
ВЫВЕСТИ " "
```

```
ДЛЯ j ОТ 0 ДО edge_count-1
```

```
    ВЫВЕСТИ "e" + (j+1)
```

```
ВЫВЕСТИ перевод строки
```

```
ДЛЯ i ОТ 0 ДО n-1
```

```
    ВЫВЕСТИ "v" + (i+1)
```

```
    ДЛЯ j ОТ 0 ДО edge_count-1
        ВЫВЕСТИ matrix[i][j]
    ВЫВЕСТИ перевод строки
```

```
ФУНКЦИЯ calculateDegree(n: ЦЕЛОЕ, edge_count: ЦЕЛОЕ, matrix: ЦЕЛОЙ[][] , vertex: ЦЕЛОЕ): ЦЕЛОЕ
    degree = 0
    ДЛЯ j ОТ 0 ДО edge_count-1
        degree = degree + matrix[vertex][j]
    ВЕРНУТЬ degree
```

```
ФУНКЦИЯ graphSize(edge_count: ЦЕЛОЕ): ЦЕЛОЕ
    ВЕРНУТЬ edge_count
```

```
ФУНКЦИЯ findIsolatedVertices(n: ЦЕЛОЕ, edge_count: ЦЕЛОЕ, matrix: ЦЕЛОЙ[][] )
    ВЫВЕСТИ "Изолированные вершины (степень 0): "
    found = ЛОЖЬ
    ДЛЯ i ОТ 0 ДО n-1
        ЕСЛИ calculateDegree(n, edge_count, matrix, i) == 0 ТО
            ВЫВЕСТИ (i+1)
            found = ИСТИНА
    ЕСЛИ НЕ found ТО ВЫВЕСТИ "нет"
    ВЫВЕСТИ перевод строки
```

```
ФУНКЦИЯ findEndVertices(n: ЦЕЛОЕ, edge_count: ЦЕЛОЕ, matrix: ЦЕЛОЙ[][] )
    ВЫВЕСТИ "Концевые вершины (степень 1): "
    found = ЛОЖЬ
    ДЛЯ i ОТ 0 ДО n-1
        ЕСЛИ calculateDegree(n, edge_count, matrix, i) == 1 ТО
            ВЫВЕСТИ (i+1)
            found = ИСТИНА
    ЕСЛИ НЕ found ТО ВЫВЕСТИ "нет"
    ВЫВЕСТИ перевод строки
```

```
ФУНКЦИЯ findDominatingVertices(n: ЦЕЛОЕ, edge_count: ЦЕЛОЕ, matrix: ЦЕЛОЙ[][] )
    ВЫВЕСТИ "Доминирующие вершины (степень " + (n-1) + "): "
    found = ЛОЖЬ
    ДЛЯ i ОТ 0 ДО n-1
        ЕСЛИ calculateDegree(n, edge_count, matrix, i) == n-1 ТО
            ВЫВЕСТИ (i+1)
            found = ИСТИНА
    ЕСЛИ НЕ found ТО ВЫВЕСТИ "нет"
    ВЫВЕСТИ перевод строки
```

```
ФУНКЦИЯ printVertexDegrees(n: ЦЕЛОЕ, edge_count: ЦЕЛОЕ, matrix: ЦЕЛОЙ[][] )
    ВЫВЕСТИ "Степени вершин:"
    ДЛЯ i ОТ 0 ДО n-1
        ВЫВЕСТИ "Вершина " + (i+1) + ": степень " + calculateDegree(n, edge_count, matrix, i)
```

```
ОСНОВНАЯ ПРОГРАММА
    УСТАНОВИТЬ случайный генератор
    УСТАНОВИТЬ локаль
```

```
    ВВЕСТИ n С КЛАВИАТУРЫ
```

```
    inc_matrix = generateIncidenceMatrix(n, edge_count)
```

ВЫВЕСТИ пустую строку
printIncidenceMatrix(n, edge_count, inc_matrix)

size = graphSize(edge_count)
ВЫВЕСТИ "Размер графа (количество ребер): " + size

ВЫВЕСТИ пустую строку
printVertexDegrees(n, edge_count, inc_matrix)

ВЫВЕСТИ пустую строку
findIsolatedVertices(n, edge_count, inc_matrix)
findEndVertices(n, edge_count, inc_matrix)
findDominatingVertices(n, edge_count, inc_matrix)

// Освобождение памяти
ДЛЯ i ОТ 0 ДО n-1
 ОСВОБОДИТЬ ПАМЯТЬ(inc_matrix[i])
ОСВОБОДИТЬ ПАМЯТЬ(inc_matrix)

ВЕРНУТЬ 0

КОНЕЦ ПРОГРАММЫ

Результаты работы программы

1-ое задание

Введите количество вершин графа: 5

Матрица смежности:

	1	2	3	4	5
1	0	0	0	1	0
2	0	1	0	0	0
3	0	0	0	1	1
4	1	0	1	1	1
5	0	0	1	1	1

Размер графа (количество ребер): 4

Степени вершин:

Вершина 1: степень 1

Вершина 2: степень 1

Вершина 3: степень 2

Вершина 4: степень 4

Вершина 5: степень 3

Изолированные вершины (степень 0): нет

Концевые вершины (степень 1): 1 2

Доминирующие вершины (степень 4): 4

E:\proekt\lab5\x64\Debug\lab5.exe (процесс 9856) завершил работу с кодом 0 (0x0).

Нажмите любую клавишу, чтобы закрыть это окно:

2-ое задание

Введите количество вершин графа: 5

Матрица инцидентности:

	e1	e2	e3	e4
v1	1	0	0	0
v2	0	1	1	0
v3	0	0	0	0
v4	0	1	0	1
v5	1	0	1	1

Размер графа (количество ребер): 4

Степени вершин:

Вершина 1: степень 1

Вершина 2: степень 2

Вершина 3: степень 0

Вершина 4: степень 2

Вершина 5: степень 3

Изолированные вершины (степень 0): 3

Концевые вершины (степень 1): 1

Доминирующие вершины (степень 4): нет

E:\proekt\lab5\x64\Debug\lab5.exe (процесс 592) завершил работу с кодом 0 (0x0).

Нажмите любую клавишу, чтобы закрыть это окно:

Вывод

В ходе выполнения данной лабораторной работы были успешно реализованы и проанализированы два основных способа представления неориентированных графов: с использованием матрицы смежности и матрицы инцидентности. Работа позволила на практике изучить особенности каждого из представлений и их применение для решения задач анализа структуры графов.

Особенностью практической реализации стало использование динамических структур данных для работы с матрицами, что позволило обрабатывать графы с произвольным количеством вершин. Как видно из результатов работы программы, представленных в отчете, оба метода дают идентичные результаты при определении степеней вершин и классификации их на изолированные (степень 0), концевые (степень 1) и доминирующие (степень $n-1$).

Важным аспектом работы стала демонстрация того, что различные математические представления графов не влияют на результат анализа, а лишь предлагают альтернативные способы хранения и обработки информации о структуре графа. Это подтверждается совпадением всех вычисляемых параметров графа в обоих заданиях, что хорошо видно на приведенных в отчете скриншотах результатов работы программы.

Полученные навыки работы с матричными представлениями графов, динамическим распределением памяти и алгоритмами анализа графовых структур являются фундаментальными для дальнейшего изучения более сложных задач теории графов и алгоритмизации, таких как поиск путей, определение связности и другие алгоритмы на графах.